

D197

Version Control

Armored Drill

8 concepts · 12 items · 4 transfer items

What this is. Every concept in this course's public description, re-asked with more disguise than a practice test uses. Each stem carries facts that do not decide it. Every wrong answer is a named trap.

Source. Scope from the course's public description in the WGU Institutional Catalog (September 2026); items written from the field's standard body of knowledge. No WGU assessment item, course material or proprietary content is used or reproduced. Not affiliated with or endorsed by Western Governors University.

Method © 2026 Erik Daniel Lindquist — Course Mastery Coach, armored drill. Items drafted with AI assistance under his direction.

How to run it

Paper first. Pencil. One pass, no notes, a timer if you want the pressure.

1. **Before you read the options**, fill the strip under the item: the target word (what is actually being asked), and the one fact in the stem that decides it.
2. **Principle before arithmetic**. Name the rule, then compute.
3. **Camouflage**. Most of every stem does not matter. Ask: which single fact decides this?
4. **Inverse pairs are the bait**. If two options are opposites, one of them is the trap built for you.
5. **Mark sure, unsure or guess** honestly. Sure-and-wrong is the most valuable miss on the page — it is a wrong mental model.
6. **Absolutes** (always, never, guarantee) are usually wrong. And recognizing an answer is not the same as owning it.
7. Items with a **heavy black bar** on the left are **transfer items**: the same principle, reversed or solved for a different variable.
8. Stop at the STOP page. Then grade, fill the triage grid, and tally the traps you fell for. The biggest tally is what you drill next.

The strip, filled in, looks like this:

Target word _____	The one fact that decides _____	Answer _____	[] sure [] unsure [] guess
-----------------------------	---	------------------------	----------------------------------

Items

1.

A developer on a team of eight at a logistics startup, working in VS Code, edits three files. She runs `git add` on two of them, then `git commit -m "fix totals"`. What does the new commit contain?

- A. The two files, and the third file's edits are discarded
- B. Changes to all three files
- C. Changes to the two staged files only
- D. Nothing, until she pushes

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

2. [TRANSFER — reversed stem / different variable]

`git status` lists `app.py` under both "Changes to be committed" and "Changes not staged for commit." What happened?

- A. The file was edited, staged, then edited again.
- B. It was committed but not pushed.
- C. It has a merge conflict.
- D. Git is broken, because a file can only ever be in one state.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

3.

A new hire needs a working copy of the team's 2 GB repository, hosted on a company server in Chicago, with the full commit history. What should she run?

- A. `git clone` with the repository URL
- B. `git init`
- C. `git pull`
- D. Download a ZIP of the files

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

4.

A developer on a team spread across three time zones wants to see the commits teammates pushed overnight without changing any files in her working directory. Which command fits?

- A. git push
- B. git fetch
- C. git pull
- D. git clone into a new folder

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

5. [TRANSFER — reversed stem / different variable]

git pull is the same as running which two commands in order?

- A. git clone, then git merge
- B. git commit, then git push
- C. git fetch, then git push
- D. git fetch, then git merge

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

6.

A developer on an app with 40,000 users needs to start a login feature without affecting main. She wants to create a branch and begin working on it at once. Which command does both?

- A. git switch -c feature/login
- B. git branch feature/login
- C. git clone feature/login
- D. git commit to main and branch later

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

7.

Merging feature into main stops with a conflict in config.yml, a 300-line file edited by two developers. Both branches changed line 12. What completes the merge correctly?

- A. Remove the markers but skip git add
- B. git push --force to override the conflict
- C. Edit line 12 to the right content, remove the markers, git add the file, then commit.
- D. Commit right away; Git keeps the newer change.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

8.

On a Friday afternoon before a Monday release, a developer's git push is rejected with "non-fast-forward." A teammate pushed two commits an hour earlier. What should she do?

- A. Delete her local repository and re-clone
- B. Pull to integrate the teammate's commits, resolve any conflicts, then push.
- C. Make another commit and push again
- D. git push --force so the remote matches her branch

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

9.

A commit that changed 14 files broke checkout on main. It was pushed yesterday, and five teammates have already pulled it. What is the safest way to undo it?

- A. git checkout the previous commit
- B. Delete the repository, since pushed commits can never be undone
- C. git revert the commit and push the new commit
- D. git reset --hard before it and force-push

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

10. [TRANSFER — reversed stem / different variable]

A developer made two local commits that were never pushed and wants them gone entirely. Which command fits?

- A. git push
- B. git reset --hard HEAD~2
- C. git reset --hard HEAD~1
- D. git revert HEAD~1..HEAD

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

11.

A team adds *.log to .gitignore, but app.log, which grows about 50 MB a day and was committed last month, still shows as modified. Why?

- A. The file is too large for Git to ignore.
- B. .gitignore works only after it is pushed.
- C. The pattern *.log is written wrong.
- D. .gitignore does not affect tracked files; run git rm --cached app.log.

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	___	

12. [TRANSFER — reversed stem / different variable]

A repo's .gitignore contains *.env. Which file will Git stop from being added?

- A.** env.txt
- B.** Every .env file in every repository on the machine
- C.** config.env, committed last month
- D.** A new, untracked file named prod.env

Target word	The one fact that decides	Answer	[] sure [] unsure [] guess
_____	_____	_____	

STOP

Answers and dissection follow. Finish every item and every strip first.

Key and dissection

Struck-through text is the camouflage: facts that do not decide the item.

1. Answer C Concept 1

A developer ~~on a team of eight at a logistics startup, working in VS Code,~~ edits three files. She runs git add on two of them, then git commit -m "fix totals". What does the new commit contain?

Target word: commit contents **Decides:** Only staged changes are committed.

A	Inverted operation	Unstaged edits stay in the working tree.
B	Dropped element	Ignores the staging step.
C	Correct	Commit records the staging area.
D	Adjacent category	Commit is local; push only shares it.

2. Answer A Concept 1 · transfer

git status lists app.py under both "Changes to be committed" and "Changes not staged for commit." What happened?

Target word: explain the two states **Decides:** Staging captures a snapshot at the time of git add.

A	Correct	The first edit is staged; the later edit is not.
B	Adjacent category	Committed changes do not appear in either list.
C	Confusion set	Conflicts show as "unmerged paths."
D	Absolute language	A file can have staged and unstaged changes at once.

3. Answer A Concept 2

A new hire needs a working copy of the team's ~~2-GB repository, hosted on a company server in Chicago,~~ with the full commit history. What should she run?

Target word: get a copy with history **Decides:** First-time copy from a remote.

A	Correct	Copies all history and sets up origin.
B	Adjacent category	Makes a new, empty repository.
C	Confusion set	Needs an existing local repository.
D	Dropped element	No history and no remote link.

4. Answer B Concept 3

A developer ~~on a team spread across three time zones~~ wants to see the commits teammates pushed overnight without changing any files in her working directory. Which command fits?

Target word: download without merging **Decides:** Must not change working files.

A	Inverted operation	Sends her commits instead of getting theirs.
B	Correct	Updates remote-tracking branches only.
C	Confusion set	Also merges, changing her files.
D	Adjacent category	Works, but makes a whole new copy instead of updating hers.

5. Answer D Concept 3 · transfer

git pull is the same as running which two commands in order?

Target word: decompose pull **Decides:** Pull downloads and integrates.

A	Adjacent category	Clone makes a new repository.
B	Confusion set	That is how you share, not receive.
C	Inverted operation	Push sends; pull does not.
D	Correct	Default pull behavior.

6. Answer A Concept 4

~~A developer on an app with 40,000 users~~ needs to start a login feature without affecting main. She wants to create a branch and begin working on it at once. Which command does both?

Target word: create and switch **Decides:** Must create and move to the branch.

A	Correct	Creates the branch and switches to it.
B	Dropped element	Creates it but leaves her on main.
C	Adjacent category	Clone copies repositories, not branches.
D	Inverted operation	Puts the work on main first.

7. Answer C Concept 5

Merging feature into main stops with a conflict in config.yml, ~~a 300-line file edited by two developers~~. Both branches changed line 12. What completes the merge correctly?

Target word: resolve a conflict **Decides:** Git cannot choose; a human must.

A	Dropped element	The file stays unmerged until staged.
B	Adjacent category	Pushing does not resolve a local merge.
C	Correct	That marks it resolved and finishes the merge.
D	Confusion set	Git never picks a side for you.

8. Answer B Concept 6

~~On a Friday afternoon before a Monday release,~~ a developer's git push is rejected with "non-fast-forward." A teammate pushed two commits an hour earlier. What should she do?

Target word: fix a rejected push **Decides:** The remote has commits she lacks.

A	Dropped element	Loses her unpushed work.
B	Correct	Her branch must contain the remote commits first.
C	Confusion set	Still missing the remote commits.
D	Inverted operation	Wipes the teammate's commits instead of adding them.

9. Answer C Concept 7

A commit ~~that changed 14 files~~ broke checkout on main. It was pushed yesterday, and five teammates have already pulled it. What is the safest way to undo it?

Target word: undo on shared history **Decides:** Already pushed and pulled.

A	Adjacent category	Only moves HEAD; main still has the bad commit.
B	Absolute language	Revert exists to undo pushed commits.
C	Correct	Adds an undo commit without rewriting history.
D	Confusion set	Rewrites shared history and breaks teammates' copies.

10. Answer B Concept 7 • transfer

A developer made two local commits that were never pushed and wants them gone entirely. Which command fits?

Target word: undo unpushed commits **Decides:** Local only, so rewriting history is safe.

A	Confusion set	Shares them instead of removing them.
B	Correct	Moves the branch back two commits.
C	Dropped element	Removes only one commit.
D	Adjacent category	Adds undo commits instead of removing them.

11. Answer D Concept 8

A team adds *.log to .gitignore, but app.log, ~~which grows about 50 MB a day~~ and was committed last month, still shows as modified. Why?

Target word: why it is still tracked **Decides:** app.log was already tracked.

A	Camouflage	Size has nothing to do with ignore rules.
B	Confusion set	It works locally right away.
C	Adjacent category	The pattern is valid.
D	Correct	Ignore rules apply only to untracked files.

12. Answer D Concept 8 • transfer

A repo's .gitignore contains *.env. Which file will Git stop from being added?

Target word: what the rule covers **Decides:** Untracked files matching the pattern.

A	Adjacent category	Does not end in .env.
B	Absolute language	A repo's .gitignore applies only to that repo.
C	Confusion set	Already tracked, so still tracked.
D	Correct	Matches and is untracked.

Triage grid

Write each item number in the box it belongs to.

	Right	Wrong
Sure	Owned.	Concept inversion — top priority. A wrong mental model; it survives rewording.
Unsure / guess	Recognition, not mastery. Re-drill with the transfer.	Not learned yet.

Trap tally

For every miss, find the letter you chose in the key and tally its trap. The biggest number is the pattern to drill.

Trap	What it does	In this drill	Your misses
Dropped element	Leaves out a required term or step.	6	
Wrong stem number	A figure copied from the prompt that is not the answer.	0	
Adjacent category	The right idea in the wrong bucket.	11	
Inverted operation	Direction flipped: add/subtract, cause/effect, higher/lower.	5	
Absolute language	Always, never, guarantee — where judgment applies.	3	
Confusion set	Every option is real; the test is exact role-matching.	10	
Camouflage	Answers a stem fact that was never the question.	1	

Anchor rows — the concept list

One row per concept: the rule to lock in, the trap that catches people, and the tell in the question that signals it.

#	Concept	Rule	Trap	Tell
1	Working tree, staging area, repository	git add copies changes to the staging area; git commit records only what is staged.	Thinking commit saves every edited file.	Some files added, then a commit.
2	Cloning	git clone copies a remote repository, including its full history, to a new local folder and sets up the remote.	Using init or a ZIP download to get a working copy with history.	A new team member needs the project for the first time.
3	Fetch vs. pull	git fetch downloads remote commits without changing your files; git pull = fetch + merge (or rebase).	Using pull when you only want to look.	"See what changed without touching my work."
4	Branching	A branch is a movable pointer; git switch -c (or checkout -b) creates it and moves you onto it.	Creating a branch with git branch and then committing on main.	Start isolated work on a new feature.
5	Merge conflicts	When both branches change the same lines, Git stops; you edit the file, git add it, and commit to finish.	Expecting Git to pick a winner, or forcing a push.	Conflict markers after a merge.
6	Rejected pushes	A push is rejected when the remote has commits you lack; integrate them first (pull), then push.	Force-pushing and wiping teammates' commits.	"Updates were rejected ... non-fast-forward."
7	Revert vs. reset	git revert adds a new commit that undoes an old one, safe for shared history; git reset moves the branch pointer and rewrites history, fit only for unpushed work.	Resetting and force-pushing a shared branch.	Undo a commit that is already pushed vs. only local.
8	.gitignore scope	.gitignore only stops untracked files from being added; files already tracked must be removed with git rm --cached.	Expecting .gitignore to untrack committed files.	A file listed in .gitignore still shows changes.

Course scope (public description): Version control is critical to maintaining software and enabling scalability solutions. A best practice for any programming project that requires multiple files uses version control. Version control enables teams to have collaborative workflows and enhances the software development lifecycle. This course introduces students to the basics of publishing, retrieving, branching, and cloning. There are no prerequisites for this course.